

30. dashboard + HTTP API

solar_ws0129-main

2026-07-29

対象

項目	内容
ファイル	mpc_solarcar/dashboard_node.py
種別	Python
区分	runtime node
役割	planner と vehicle の現在値を ROS から受け、HTTP サーバと dashboard frontend へ渡す。
起動文脈	可視化の中心ノード。

このファイルを読む理由

planner と vehicle の現在値を ROS から受け、HTTP サーバと dashboard frontend へ渡す。

- /api/state と /metrics を持つ。
- ROS topic を直接 browser へ出すのではなく、Node 内 state に集約する。

呼び出し関係

どこから呼ばれるか

- launch/solarcar_sim.launch.py
- mpc_solarcar/live_launch.py
- launch/solar_measurement.launch.py

次に読むと理解が進むファイル

- 特記事項なし。

同一リポジトリ内の主要依存

- 同一リポジトリ内の明示 import は少ない。

ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

代表コード断片

```
from ament_index_python.packages import get_package_share_directory

class DashboardHandler(SimpleHTTPRequestHandler):
    def __init__(self, *args, node=None, directory=None, **kwargs):
        self._node = node
        super().__init__(*args, directory=directory, **kwargs)

    def do_GET(self):
        if self.path.startswith('/api/state'):
            self._send_json(self._node.get_state())
            return
        if self.path.startswith('/metrics'):
            self._send_metrics(self._node.get_prometheus_metrics())
            return
        super().do_GET()

    def log_message(self, format, *args):
        # quiet
        return

    def _send_json(self, data):
```

主要構造の読み取り

主要クラスは DashboardHandler, DashboardNode。主要関数は do_GET, log_message, destroy_node, get_state, get_prometheus_metrics, main。ROS パラメータ宣言は 5 件。ROS I/O は publisher 0 件、subscription 21 件。

主要クラス・関数

行	種別	名前	補足
19	class	DashboardHandler	bases: SimpleHTTPRequestHandler
55	class	DashboardNode	bases: Node
20	function	__init__	args: self
24	function	do_GET	args: self
33	function	log_message	args: self, format
37	function	_send_json	args: self, data
45	function	_send_metrics	args: self, payload
56	function	__init__	args: self
156	function	destroy_node	args: self
165	function	get_state	args: self
169	function	get_prometheus_metrics	args: self
236	function	_update	args: self, key, value
241	function	_on_speed_cmd	args: self, msg
244	function	_on_upper_speed_cmd	args: self, msg
247	function	_on_speed_meas	args: self, msg
250	function	_on_throttle_cmd	args: self, msg
253	function	_on_drive_mode	args: self, msg

256	function	<code>_on_soc</code>	args: self, msg
264	function	<code>_set_estimated_soc</code>	args: self, value
273	function	<code>_on_tb</code>	args: self, msg
276	function	<code>_on_ibatt</code>	args: self, msg
279	function	<code>_on_vbatt</code>	args: self, msg
282	function	<code>_on_s_km</code>	args: self, msg
285	function	<code>_on_status</code>	args: self, msg
301	function	<code>_on_env</code>	args: self, msg
313	function	<code>_on_metrics</code>	args: self, msg
333	function	<code>_on_upper_plan</code>	args: self, msg
344	function	<code>_on_lower_plan</code>	args: self, msg
355	function	<code>_on_sys_state</code>	args: self, msg
358	function	<code>_on_sys_diag</code>	args: self, msg
361	function	<code>_on_mpc_state</code>	args: self, msg
364	function	<code>_on_health</code>	args: self, msg
367	function	<code>_on_gps</code>	args: self, msg
373	function	<code>_on_path</code>	args: self, msg
379	function	<code>_init_dummy</code>	args: self, path, rate_hz
395	function	<code>_tick_dummy</code>	args: self
409	function	<code>main</code>	

ファイルを上から読んだときの定義順

行	内容
19	クラス <code>DashboardHandler</code> を定義する。
55	クラス <code>DashboardNode</code> を定義する。
409	関数 <code>main</code> を定義する。
417	条件 <code>__name__ == '__main__'</code> を判定し、真なら内部処理を行う。

import 群の詳細

行	import 文	このファイルで読む理由
1	<code>import json</code>	<code>manifest</code> 、 <code>checkpoint</code> 、 <code>UDP payload</code> をやり取りするため。このファイル内での主な使用位置は L38。
2	<code>import math</code>	<code>clip</code> 、 <code>sqrt</code> 、 <code>sin/cos</code> 、有限判定など数値ロジックに使うため。このファイル内での主な使用位置は L212。
3	<code>import os</code>	パス、環境変数、プロセス外部状態を扱うため。このファイル内での主な使用位置は L68, L70, L71。
4	<code>import threading</code>	<code>threading</code> モジュールを利用するため。このファイル内での主な使用位置は L73, L150。
5	<code>import time</code>	<code>wall/monotonic time</code> に基づく周期制御や <code>freshness</code> 判定を行うため。このファイル内での主な使用位置は L75, L215, L239, L261, L262, L268, L299, L311, ...。
6	<code>from functools import partial</code>	<code>functools</code> から <code>partial</code> を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L147。

7	<code>from http.server import SimpleHTTPRequestHandler, ThreadingHTTPServer</code>	<code>http.server</code> から <code>SimpleHTTPRequestHandler</code> , <code>ThreadingHTTPServer</code> を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L19, L149。
9	<code>import rclpy</code>	ROS 2 Python ノードとして起動・spin するため。このファイル内での主な使用位置は L410, L412, L414。
10	<code>from rclpy.node import Node</code>	ROS 2 ノード本体の基底クラスとして使うため。このファイル内での主な使用位置は L55。
12	<code>from std_msgs.msg import Float32, Float32MultiArray, String</code>	<code>Float32</code> や <code>String</code> など軽量メッセージで <code>planner / vehicle</code> 値を <code>publish</code> するため。このファイル内での主な使用位置は L117, L118, L119, L120, L121, L122, L123, L124, ...。
13	<code>from sensor_msgs.msg import NavSatFix</code>	GPS 緯度経度高度を ROS topic でやり取りするため。このファイル内での主な使用位置は L136, L367。
14	<code>from nav_msgs.msg import Path</code>	将来軌跡を <code>dashboard</code> や <code>logger</code> へ出すため。このファイル内での主な使用位置は L137, L373。
16	<code>from ament_index_python.packages import get_package_share_directory</code>	<code>ament_index_python.packages</code> から <code>get_package_share_directory</code> を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L67。
381	<code>import pandas as pd</code>	CSV や時刻列の読込・整形・再サンプリングに使うため。このファイル内での主な使用位置は L382。

関数・クラスを上から順に解説

L19 クラス `DashboardHandler`

- 定義: `DashboardHandler(bases=SimpleHTTPRequestHandler)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `__init__`, `_send_json`, `_send_metrics`, `do_GET`, `dumps`, `encode`, `end_headers`, `get_prometheus_metrics`, `get_state`, `len`, `send_header`, `send_response`
 - 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
1. 関数 `__init__` を定義する。
 2. 関数 `do_GET` を定義する。
 3. 関数 `log_message` を定義する。
 4. 関数 `_send_json` を定義する。
 5. 関数 `_send_metrics` を定義する。

L55 クラス `DashboardNode`

- 定義: `DashboardNode(bases=Node)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Lock`, `Thread`, `ThreadingHTTPServer`, `__init__`, `_init_dummy`, `_set_estimated_soc`, `_update`, `abspath`, `create_subscription`, `create_timer`, `declare_parameter`, `destroy_node`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。

1. 関数 `__init__` を定義する。
2. 関数 `destroy_node` を定義する。
3. 関数 `get_state` を定義する。
4. 関数 `get_prometheus_metrics` を定義する。
5. 関数 `_update` を定義する。
6. 関数 `_on_speed_cmd` を定義する。
7. 関数 `_on_upper_speed_cmd` を定義する。
8. 関数 `_on_speed_meas` を定義する。
9. 関数 `_on_throttle_cmd` を定義する。
10. 関数 `_on_drive_mode` を定義する。
11. 関数 `_on_soc` を定義する。
12. 関数 `_set_estimated_soc` を定義する。

L409 関数 `main`

- 定義: `main()`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `DashboardNode`, `destroy_node`, `init`, `shutdown`, `spin`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。

1. `rcipy.init(...)` を実行する。
2. `node` に `DashboardNode()` の結果を代入する。
3. `rcipy.spin(...)` を実行する。
4. `node.destroy_node(...)` を実行する。
5. `rcipy.shutdown(...)` を実行する。

設定値と入力パラメータ

行	名前	既定値・補足
58	<code>host</code>	0.0.0.0
59	<code>port</code>	8080
60	<code>static_dir</code>	
61	<code>dummy_csv</code>	
62	<code>dummy_rate_hz</code>	5.0

ROS topic I/O

行	種別	topic	callback / 補足
117	subscription	/planner/speed_cmd	self._on_speed_cmd
118	subscription	/planner/upper_speed_cmd	self._on_upper_speed_cmd
119	subscription	/vehicle/speed_kmh	self._on_speed_meas
120	subscription	/planner/throttle_cmd_pct	self._on_throttle_cmd
121	subscription	/planner/drive_mode	self._on_drive_mode
122	subscription	/vehicle/batt_soc	self._on_soc
123	subscription	/vehicle/batt_temp_c	self._on_tb
124	subscription	/vehicle/batt_current_a	self._on_ibatt
125	subscription	/vehicle/batt_voltage_v	self._on_vbatt
126	subscription	/vehicle/s_kmh	self._on_s_kmh
127	subscription	/planner/status	self._on_status
128	subscription	/planner/env	self._on_env
129	subscription	/planner/metrics	self._on_metrics
130	subscription	/planner/upper_plan	self._on_upper_plan
131	subscription	/planner/lower_plan	self._on_lower_plan
132	subscription	/system/state	self._on_sys_state
133	subscription	/system/diag	self._on_sys_diag
134	subscription	/system/mpc_state	self._on_mpc_state
135	subscription	/system/health	self._on_health
136	subscription	/sim/gps	self._on_gps
137	subscription	/planner/trajectory	self._on_path

処理の流れ

1. 初期化時に設定値や入力パスを読み込む。
2. publisher / subscription / timer を準備する。
3. timer callback 周期で主処理を進める。

このファイルの位置づけ

この資料群では、`mpc_solarcar/dashboard_node.py` を **runtime node** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。